

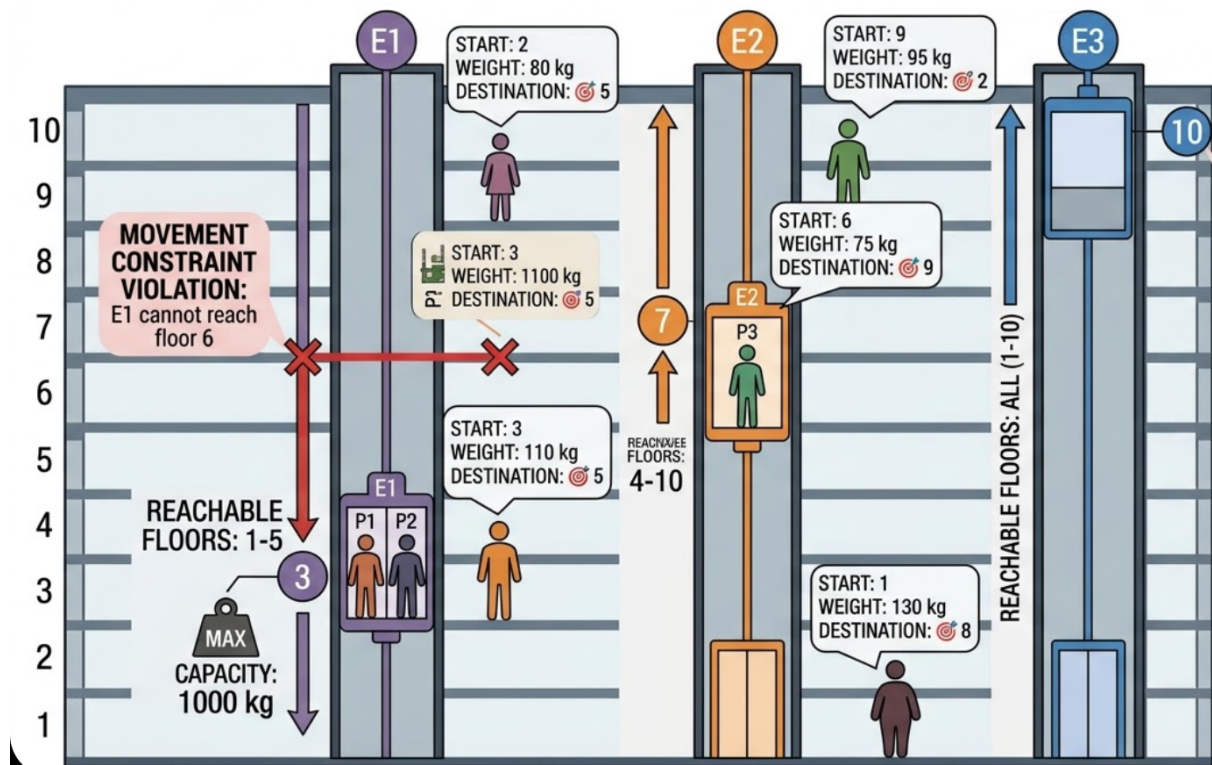
Assignment 1: Multi-Elevator Passenger Planning

Bar-Ilan CS 89-570
Introduction to Artificial Intelligence

April 16, 2026

1 Introduction

In this exercise you will *experiment with solving a planning/path-finding problem* using deterministic search algorithms we studied (A*).



Summary of the Domain In this problem, several elevators move inside a building and transport people to their destination floors. Each elevator has its own current floor, a set of floors it can reach, and a maximum weight capacity. Each person has a starting floor, a weight, and a destination floor. The task is to plan a shortest valid sequence of actions that brings all people to their goal floors.

This domain combines *movement planning* with *resource constraints*: an elevator may move only to floors it can reach, and the total weight of the people inside it may not exceed its maximum weight.

2 Problem Description

The building contains floors numbered from 0 up to **height**. Floor 0 is the ground floor. A problem instance contains a set of elevators and a set of persons.

Elevators You control a set of elevators. Each elevator has a unique ID number and three properties:

- its current floor,
- the set of floors it can reach,
- its maximum weight capacity.

Persons Each person has a unique ID number and three properties:

- a starting floor,
- a weight,
- a destination floor.

A person may either stand on some floor or be inside exactly one elevator.

Actions Each step applies exactly one action. There are three types of actions:

- `MOVE(ELEVATOR_ID, TARGET_FLOOR)` – move an elevator from its current floor to another floor that belongs to the set of floors this elevator can reach.
- `ENTER(PERSON_ID, ELEVATOR_ID)` – move a person from the current floor into an elevator. This is legal only if the person and the elevator are on the same floor, the person is not already inside an elevator, and after entering, the total weight inside the elevator does not exceed the elevator's maximum weight.
- `EXIT(PERSON_ID, ELEVATOR_ID)` – move a person out of an elevator to the elevator's current floor. This is legal only if the person is currently inside that elevator.

At each time step, exactly one action is executed.

Action representation (string format). Let e denote an **elevator ID**, p a **person ID**, and f a **floor number**. Each action must be encoded as one of the following strings:

- `MOVE(E, F): MOVE{e,f}`
- `ENTER(P, E): ENTER{p,e}`
- `EXIT(P, E): EXIT{p,e}`

For example, if elevator 2 moves to floor 5, then person 7 enters elevator 2, and later exits from it, the output should be: `[MOVE{2,5}, ENTER{7,2}, EXIT{7,2}]`.

In the successor function, each child is represented as `(action, next_state)`, and you should store this string in the `action` field.

Additional domain rules

- An elevator may move only to floors that appear in its own reachable-floor set.
- Persons do not change elevators automatically. They move only through explicit ENTER and EXIT actions.
- The weight limit must always be respected.
- A person cannot be inside two elevators at the same time.
- A person already standing on a floor cannot execute EXIT; a person already inside an elevator cannot execute ENTER again.

3 Initial State and the Goal

Initial state representation The initial state of the problem is represented as a Python dictionary with the following entries:

1. **height** – an integer. The building floors are $\{0, 1, \dots, \text{height}\}$.
2. **Elevators** – a dictionary whose keys are elevator IDs. The value of elevator ID e is a tuple

$$(f, F, w_{\max}),$$

where f is the current floor of the elevator, F is the set of floors this elevator can reach, and $w_{\max}$ is its maximum weight capacity.

3. **Persons** – a dictionary whose keys are person IDs. The value of person ID p is a tuple

$$(f_{\text{start}}, w, f_{\text{goal}}),$$

where f_{start} is the person's starting floor, w is the person's weight, and f_{goal} is the destination floor.

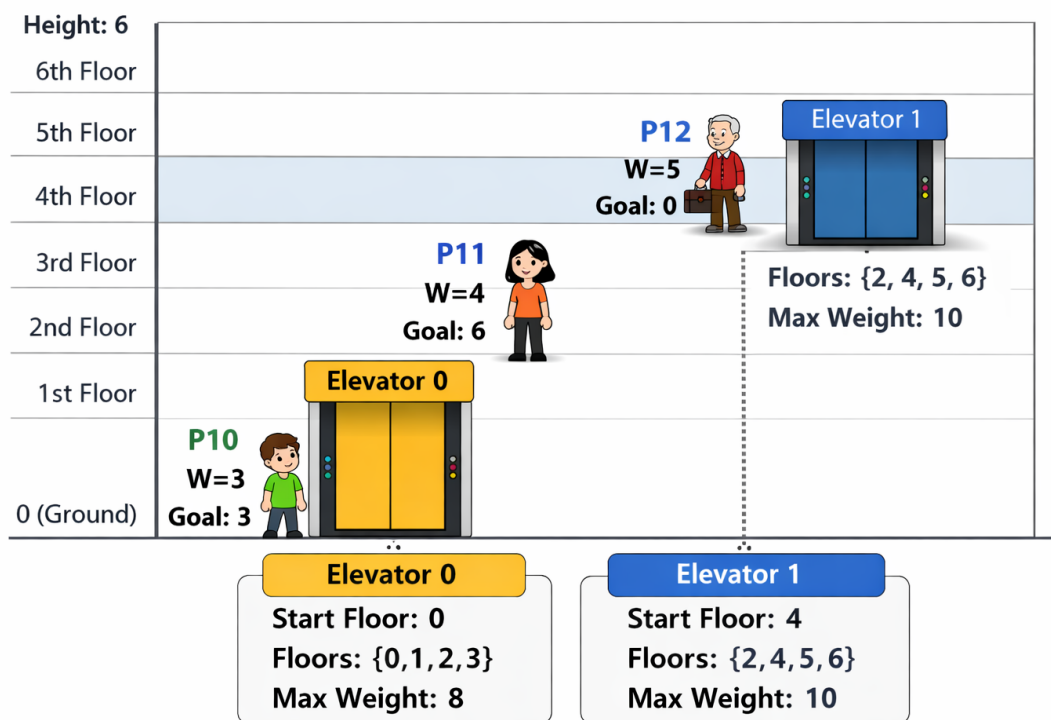
A state must additionally encode, for each person, whether the person is currently standing on a floor or riding inside an elevator. You are free to choose the exact internal representation, as long as it is consistent and hashable.

Goal description The problem is **SOLVED** if every person is at their destination floor and is not inside an elevator.

4 Example

```
init_state_p1 = {
  "height": 6,
  "Elevators": {
    0: (0, (0, 1, 2, 3), 8),
    1: (4, (2, 4, 5, 6), 10)
  },
  "Persons": {
    10: (0, 3, 3),
    11: (2, 4, 6),
    12: (4, 5, 0)
  }
}
```

Example Initial State



5 Tips

We provide a starting state `init_state`. Each student may represent the state however they like, but once you define the state representation in the Python initialization function, you must keep using the same representation for every state you create in the successor function.

Note that each state must be *hashable*, i.e., all its components must be immutable. Examples of immutable structures in Python include: tuple, string, and frozenset. You can also make hashable classes, but then you need to implement `__eq__` and `__hash__`.

- Keep the **A* heuristic admissible**. For example, the blind heuristic is admissible but weak. The real challenge is to make it informative and still admissible.
- Handle **capacity constraints** correctly.
- Handle **elevator reachability sets** correctly. Not every elevator can visit every floor.
- Generate only **legal actions**. This will be checked.
- Keep the **successor function** as small and efficient as possible, to avoid redundant successors.
- Note that while LLMs can help you brainstorm heuristics, they cannot prove that a heuristic is admissible.

6 Important

Since the problem can be represented in many different ways (again – you are free to choose how to represent the state and the problem), we need the action names and the IDs of the elevators/persons to be consistent among all submissions. Do not change the IDs you got as input. Use the provided action names and format. For example, the action that moves elevator 1 to floor 5 is written as `MOVE{1,5}`.

7 What You Must Implement

We provide a starter codebase. Most infrastructure is already implemented. Your job is to complete the methods so that the checker can solve instances.

Context. The state is your encoding, that you choose to represent the problem.

Files and required functions (in `ex1.py`)

You *must* implement the following (you may add helpers as you see fit):

1. `successor(state) -> List[(action, next_state)]`

- **Input:** a `state` object (hashable) representing the current problem configuration.
- **Output:** a *list* of tuples `(action, next_state)`.
- **Try to keep this list as small as possible by using pruning techniques, but do not prune any action that may be part of an optimal solution.**
- **Actions to consider (3 types):** `MOVE(ELEVATOR_ID, TARGET_FLOOR)`, `ENTER(PERSON_ID, ELEVATOR_ID)`, `EXIT(PERSON_ID, ELEVATOR_ID)`.
- **Side effects (purely functional):** `next_state` must be a *new* object with updated positions/status; do not mutate `state` in place.

2. `goal_test(state) -> bool`

- **Input:** a `state`.
- **Return:** True iff every person is located at their goal floor and is not inside any elevator.

3. `h_aster(node) -> int` (A^* heuristic; must be *admissible*)

- **Input:** a `node` whose property `node.state` holds the current state.
- **Output:** a nonnegative integer that *lower-bounds* the remaining cost to any goal.
- **Meaning:** this is the heuristic A^* uses; it must never overestimate the optimal remaining cost. (Consistency is preferred but not required.)

8 Scoring & Evaluation (How the checker works)

- There will be **10 problem instances**
- Each instance is tested once with A^* , using your implementations of `successor`, `goal_test`, and the corresponding heuristic.
- The running time of each test instance is measured. Each problem has a 30-second time limit, and solving a problem (i.e, output an optimal plan) under 30 second earns you 7 points.
- The remaining 30 points are awarded by competition. Your solution will be tested on 5–10 new instances. Strong performance in the competition may earn you more than 30 points.

What counts as “solved”? An instance is considered solved if your algorithm returns a *valid* action sequence that reaches a goal state and passes the automatic simulator checks (preconditions/effects and goal).

- **Additional requirement for A^* :** the returned plan *must be optimal* with respect to the instance’s action costs. The checker verifies optimality.

9 Attached files

The starter code includes the following files:

1. **ex1.py** — *the only file you submit and the only one graded*. Implement `successor`, `goal_test`, `h_astar`.
2. **ex1_check.py** — local *self-checker*. Use it to run your code on the provided instances. You may add *more* problems here for your own testing, but the grading system *will not* use your added problems. Changes to this file do not affect grading.
3. **utils.py** and **search.py** — auxiliary code (data structures and search drivers used by the assignment). You do not submit these files; do not rely on modifying them for your solution.

How to self-check (quick start). Run the checker from the assignment folder:

```
python ex1_check.py
```

Follow the comments inside `ex1_check.py` to select algorithms (A*/GBFS) and instances. You can run it with `python3` if you want to.

10 Rules & Submission

- **Edit only ex1.py.** Do not modify or submit any other file.
- **Submission.**
 1. Rename `ex1.py` to `ex1_id.py`, where `id` is your ID number. For example: `ex1_000000000.py`.
 2. Submit the renamed file using the Bar-Ilan submit system: <https://submit.cs.biu.ac.il/cgi-bin/welcome.cgi>
- **Individual work.** Discussing ideas is allowed, but *sharing or copying code is prohibited*.
- **Use of AI / online sources.** You may use them, but you *must* clearly state *at the top of the document* which LLMs or online sources you used and how you used them (for example: writing parts of the code, suggesting specific actions, providing ideas, or anything else). Failure to disclose this information will result in a grade reduction.
- **Academic integrity.** We take plagiarism seriously.
- **Questions.** Ask during office hours or in the assignment forum only.
- **Extensions.** No extensions will be granted except for documented exceptional circumstances via email.
- **Deadline:** 13.5.2026.